Sparse Retrieval and Hybrid Search

pig7selene

September 5, 2026

Abstract

Sparse retrieval ranks documents through observable lexical evidence, while dense retrieval ranks learned semantic representations. Neither signal is sufficient for every query. This chapter develops Bag-of-Words representations, TF-IDF, BM25, and inverted indexes, then explains how sparse and dense candidate sets can be combined through score fusion and rank fusion. It also introduces learned sparse retrieval, filtering, and operational contracts that keep a hybrid system interpretable, reproducible, and debuggable.

1 Introduction

Chapter 33 developed dense retrieval as a learned ranking function over query and document embeddings. Its strength is that a useful passage need not repeat the query's wording. That strength does not make exact lexical evidence obsolete. A model lookup for a package name, a version number, an error code, a scientific identifier, a product SKU, or a rare entity often depends on preserving precisely the terms that occur in the source. Replacing exact matching with a broad semantic signal can turn a query for one identifier into a highly ranked passage about a related but wrong identifier.

Sparse retrieval treats a query and retrieval unit as weighted collections of terms. Most possible terms have weight zero in any one collection, which makes the representation sparse and permits efficient lookup through an inverted index. The family includes classical Bag-of-Words methods such as TF-IDF and BM25 as well as learned sparse models. This chapter uses **lexical** to mean that the score retains an explicit vocabulary-aligned matching interface; it does not mean that every sparse weight is hand-designed.

Sparse and dense retrieval should therefore be viewed as complementary signals. Lexical methods can preserve rare terms and make an obvious match easy to inspect. Dense methods can recover paraphrases and vocabulary mismatch. A **Hybrid Search** system uses both paths, reconciles their candidates or rankings, and passes a broader, more robust candidate set to the later context construction and reranking stages. This chapter does not develop Cross-Encoder or LLM reranking, query rewriting, multi-query retrieval, or RAG evaluation frameworks. Those chapters consume the candidate interface established here.

2 Lexical Representations and TF-IDF

Let the collection-specific lexical vocabulary be

$$\mathcal{V}_{\text{lex}} = \left\{t_1, ..., t_{V_{\text{lex}}}\right\}. \tag{1}$$

A Bag-of-Words representation maps a document d to a vector $v(d) \in R^{V_{\text{lex}}}$, with one coordinate per term. The coordinate can record presence, raw count, or a derived weight. Because a document contains only a small subset of the full vocabulary, most coordinates are zero. Bag-of-Words deliberately retains lexical identity while largely ignoring order, syntax, and long-range compositional meaning. It is an abstraction, not a claim that word order never matters for relevance.

Let $f(t, d)$ denote the number of occurrences of term t in document d. **Term Frequency** (TF) makes this observed frequency an evidence feature. A raw count rewards repetition, but not every repeated term is informative. Terms that occur in nearly every corpus item can dominate a count while doing little to distinguish one result from another.

For a collection of N documents, let $\mathrm{df}(t)$ be the number containing t. A conceptual **Inverse Document Frequency** (IDF) is

$$\mathrm{IDF}(t) \approx \log \frac{N}{\mathrm{df}(t)}. \tag{2}$$

When $\mathrm{df}(t)$ is small, the term is comparatively rare and Equation 2 gives it greater weight. The exact smoothing convention differs across systems, especially for terms with extreme document frequencies; the stable idea is that terms common across the collection supply weaker discriminatory evidence than terms found in a small subset.

TF-IDF combines local occurrence and corpus-wide rarity:

$$w_{\text{TF-IDF}(t,d)} = \mathrm{TF}(t,d)\,\mathrm{IDF}(t). \tag{3}$$

Query and document vectors can then be compared through weighted lexical overlap, often an inner product or a normalized variant. Classical information retrieval develops this vector-space view and its assumptions in detail [1]. In a RAG system, the important boundary is practical: a TF-IDF score ranks the terms emitted by the declared analyzer. Tokenization, lowercasing, stemming, punctuation policy, synonym handling, and language analysis can change which apparent strings count as the same term.

3 BM25 and Inverted-Index Retrieval

BM25 is a widely used lexical ranking function that refines raw term counting with saturation and document-length normalization. One standard conceptual form is

$$s_{\text{BM25}(q,d)} = \sum_{t \in q} \mathrm{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{\ell_{\text{avg}}}\right)}. \tag{4}$$

Here q is the analyzed query, $|d|$ is document length under the selected analyzer, ℓ_{avg} is average document length in the indexed collection, $k_1 > 0$ controls term-frequency saturation, and b controls the strength of length normalization. A production implementation may use a different IDF smoothing rule, query-frequency treatment, or field-aware extension. The notation in Equation 4 exposes the roles that must remain explicit, not one universally mandated parameterization [2].

The numerator and denominator in the fraction make repeated occurrences valuable but not linearly valuable forever. The first few appearances of a query term can be strong evidence that a document is about that term; the hundredth repetition may be boilerplate, a list, or accidental duplication. Increasing $f(t,d)$ therefore produces diminishing incremental gain. This **term-frequency saturation** prevents repetition from overwhelming the remainder of the query evidence.

Long documents have more opportunities to contain a query term by chance. The $|d\frac{|}{\ell_{\text{avg}}}$ factor compensates for this advantage relative to an average-length unit. Larger b gives document length more influence; smaller b moves toward no such correction. The correct setting depends on the retrieval unit distribution. Chapter 35 makes this dependency concrete: a corpus with uniform token windows has a different length profile from one indexed at paragraph or section granularity.

Sparse search is efficient because it uses an **inverted index**. Instead of scanning every document for each query, the index maps a term to a posting list:

$$t \to [(\text{document ID}, f(t,d), \text{positions}, \text{metadata}), \ldots]. \tag{5}$$

Only documents appearing in posting lists for query terms need be considered for a lexical score. A posting can retain a document or chunk identifier, term frequency, positions for phrase or proximity logic, and fields needed for filtering. This is not the **Inverted File Index** (IVF) discussed in Chapter 34. Both names use "inverted," but an inverted index maps lexical terms to documents, whereas IVF maps vectors to coarse vector-space regions. Their stored objects, candidate-generation rules, and failure modes are different.

4 Sparse and Dense Signals

Sparse and dense retrievers answer different matching questions. Sparse retrieval asks whether the query's terms, or terms assigned nonzero sparse weights, appear in a unit with informative frequency. Dense retrieval asks whether independently computed embeddings occupy a compatible neighborhood under a learned similarity function. Sparse search is consequently strong for exact names, rare terminology, identifiers, version strings, and evidence that must visibly overlap. Dense search is useful when a relevant source expresses the same intent in different wording.

Signal	Often strong for	Characteristic blind spot
Sparse lexical	Rare terms, identifiers, exact quotations, and inspectable token overlap	Paraphrases, synonymy, and terminology mismatch
Dense semantic	Paraphrases, semantic relatedness, and learned query–passage associations	Fine lexical distinctions, rare strings, and semantically related but factually wrong matches
Hybrid	Robust first-stage candidate generation across both signal types	Score calibration, duplicate management, and added system cost

Table 1: Sparse and dense retrieval have complementary failure modes. A hybrid design aims to preserve candidates that either path finds persuasive, rather than declaring one signal universally sufficient.

The distinction is empirical, not categorical. A sparse query can miss "automobile maintenance" when it asks for "car repair." A dense retriever can retrieve a passage about a semantically related system that has the wrong version, numerical condition, or entity. Work comparing sparse, dense, and more expressive retrieval representations illustrates why sparse precision and dense semantic matching can be productively combined rather than treated as mutually exclusive [3].

5 Hybrid Candidate Generation and Fusion

A basic Hybrid Search pipeline runs two candidate generators in parallel:

$$\text{query} \longrightarrow \text{sparse retriever} \longrightarrow \text{sparse candidates}$$
$$\text{query} \longrightarrow \text{dense retriever} \longrightarrow \text{dense candidates}$$
$$\text{candidate union} \longrightarrow \text{fusion} \longrightarrow \text{Top-}k\text{ results} \longrightarrow \text{later reranking}$$

Let $C_{\mathrm{s}(q)}$ and $C_{\mathrm{d}(q)}$ be the candidate sets from sparse and dense paths. Their first-stage union is

$$C_{\mathrm{hybrid}(q)} = C_{\mathrm{s}(q)} \cup C_{\mathrm{d}(q)}. \tag{6}$$

The union can improve recall because a document missed by one retriever remains eligible when found by the other. It also introduces duplicates, a larger candidate set, and the need to decide how much work each path receives. Candidate depth controls both downstream recall and cost. A system that fetches a very large result set from both paths may negate first-stage latency gains or overwhelm a later reranker.

Score fusion combines numeric scores after a declared normalization or calibration. A simple weighted form is

$$s_{\mathrm{hybrid}(q,d)} = \alpha \hat{s}_{\mathrm{dense}(q,d)} + (1-\alpha)\hat{s}_{\mathrm{sparse}(q,d)}, \quad 0 \le \alpha \le 1. \tag{7}$$

The hats in Equation 7 are essential. Raw BM25 scores and dense similarities need not share a scale, range, distribution, or even a comparable notion of distance. Direct addition can make one retriever dominate merely because its numbers are larger. Per-query normalization, score calibration on held-out data, weighted fusion, and learned fusion are alternative ways to construct comparable inputs. None is automatically correct under corpus, query, model, or filter changes.

Rank fusion avoids comparing raw scores directly. Each retrieval path supplies an ordering, and a fusion function rewards documents that rank highly in one or more lists. **Reciprocal Rank Fusion** (RRF) is a compact example:

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k_{\mathrm{RRF}} + \mathrm{rank}_{r(d)}}. \tag{8}$$

Here $\mathcal{R}$ is the set of retrieval systems, $\mathrm{rank}_{r(d)}$ is the one-based rank assigned by system r, and k_{RRF} is a smoothing constant. It is not the top-k retrieval depth. A document at a high rank contributes more than one near the bottom, while a document appearing in both lists receives evidence from both. RRF was introduced as a simple rank-combination method for information retrieval systems [4]. Its rank-only nature can be attractive when score calibration is unreliable, although it discards differences in score magnitude that may be informative.

6 Learned Sparse Retrieval and Filtering

The sparse/dense distinction is not synonymous with classical/neural. A **learned sparse retriever** uses a neural model to assign vocabulary-aligned weights to a query and document while retaining sparsity and inverted-index compatibility. It can learn expansion-like associations that a raw Bag-of-Words representation lacks, yet preserve explicit terms and an efficient posting-list interface. SPLADE is a representative family: it learns sparse lexical and expansion weights under sparsity regularization for first-stage ranking [5].

Learned sparse models add their own contracts. The vocabulary, analyzer, expansion behavior, pruning rule, weight dtype, and index builder determine which nonzero dimensions exist at serving time. A model may improve a paraphrase match by activating a related term, but that activation is still a learned ranking signal rather than an explanation of truth. As with dense retrievers, a source revision requires re-representation before the index can be considered current.

Sparse, dense, and hybrid paths must apply the same eligibility policy. Date, language, tenant, document type, and access-control filters can be applied before candidate generation, after it, or through an index-specific mechanism. Chapter 34 explains the broader pre-filtering and post-filtering trade-off for vector search. The hybrid extension is simple but important: a policy filter applied to only one path changes not only security or governance semantics but also the apparent quality of the fusion procedure.

7 Pipeline Design and Failure Diagnosis

An end-to-end first stage can make its boundaries explicit:

query → lexical analysis → BM25 retrieval

query → compatible query encoder → dense ANN retrieval

candidate union → score or rank fusion → candidate set → later reranking

The lexical analyzer and dense encoder should not be silently assumed to agree. One may lowercase or stem an identifier that the other tokenizes differently; one may accept a language that the other model handles poorly. Candidate identifiers, corpus revisions, and filters provide the interface on which their results can actually be joined. Reranking, addressed next, is a later selective comparison stage rather than a reason to leave these first-stage boundaries unspecified.

Failures should be diagnosed by path. A **sparse-retrieval failure** can arise from analyzer mismatch, vocabulary mismatch, missing aliases, poor field construction, rare-term overemphasis, or unhelpful length normalization. A **dense-retrieval failure** can arise from domain mismatch, semantic confusion, an incompatible embedding/index revision, or an ANN miss. A **fusion failure** arises when the paths contain useful candidates but normalization, weights, ranks, duplication rules, or filtering discard them. Treating all three as "RAG retrieval failure" hides the repairable interface.

Other system failures are coupled to candidate size. Fetching too few items from one path limits complementarity; fetching too many can increase latency and duplicate context. Badly calibrated

scores can give one retriever permanent control. Inconsistent filtering can expose an unauthorized candidate on one path or make one path appear weak because it faced a narrower corpus. The appropriate monitoring unit is therefore a query trace that records analyzed terms, sparse postings, embedding revision, each candidate list, filter decisions, fusion inputs, and final candidate order.

8 Implementation Contracts

The **sparse contract** must name the analyzer and lexical vocabulary, text fields, normalization, stopword and stemming policy where used, posting format, BM25 or alternative scoring configuration, document-length statistic, and exact corpus revision. It should specify how an analyzed query is represented when no term has a posting list and how phrase, positional, or field behavior affects the score.

The **dense contract** must preserve the encoder, tokenizer, similarity convention, ANN index, and corpus revision from Chapters 33 and 34. The **hybrid contract** must bind the two paths to a common retrieval-unit identifier, eligibility policy, candidate depths, deduplication rule, fusion method, normalization or calibration revision, and final top-k tie rule. A result must remain traceable to the path or paths that supplied it; otherwise, a score regression cannot be assigned to sparse retrieval, dense retrieval, or fusion.

The **evaluation contract** should report sparse-only, dense-only, and hybrid candidate recall separately under the same filters and relevance judgments. It should also report candidate overlap, union size, duplicate rate, fusion ablations, latency by path, tail latency of the joined request, and downstream support coverage. Tests should include exact identifiers, paraphrases, rare entities, conflicting source revisions, filter-sensitive records, and cases where the two paths rank different valid sources. These slices turn complementarity into a measurable claim rather than a slogan.

9 Summary

Sparse retrieval preserves lexical evidence through weighted term representations and inverted indexes. TF-IDF balances local term occurrence with corpus-wide rarity; BM25 adds term-frequency saturation and document-length normalization. These methods remain valuable wherever a precise string, rare term, or interpretable overlap is part of the evidence needed for retrieval.

Dense and sparse methods fail differently. Hybrid Search forms a candidate union and combines scores or ranks through an explicitly versioned fusion rule. Score fusion requires normalization or calibration because BM25 and embedding scores are not inherently comparable; rank fusion such as RRF avoids raw-score comparison while giving up magnitude information. A reliable system evaluates sparse, dense, and fusion quality separately, applies filters consistently, and passes a well-specified candidate set to the later reranking stage.

References

[1] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[2] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

[3] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, "Sparse, Dense, and Attentional Representations for Text Retrieval," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021, doi: 10.1162/tacl_a_00369.

[4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.

[5] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 2021, pp. 2288–2292. doi: 10.1145/3404835.3463098.